

PREPARACION PARA LA PRACTICA EXPERIMENTAL UNIDAD I

HTML5, CSS3 Y JAVASCRIPT ES6+

De cero a la practica: documento unificado

Teoria, ejemplos progresivos e integracion de las 3 tecnologias

Paso 2: HTML5 → Paso 3: CSS3 → Paso 4: JavaScript → Integracion

Aplicaciones Web – Ingenieria de Software – UTEQ 2025-2026

Como usar este documento

Este documento cubre los 3 pilares del frontend web en el orden que se necesitan para la practica: primero HTML5 (la estructura), luego CSS3 (la apariencia) y finalmente JavaScript ES6+ (el comportamiento). Cada ejemplo se puede probar creando archivos `.html`, `.css` y `.js` en VS Code con Live Server, o pegando codigo en la consola del navegador (F12). La Parte IV muestra como las 3 tecnologias se conectan en un proyecto real.

Índice

I	HTML5 – Estructura y contenido (Paso 2 de la practica)	4
1.	Que es HTML y como se estructura	4
1.1.	Estructura minima de un documento HTML5	4
2.	Etiquetas semanticas HTML5	5
3.	Formularios HTML5 con validacion nativa	7
4.	Atributos ARIA y accesibilidad	9
II	CSS3 – Apariencia visual (Paso 3 de la practica)	10
5.	Como conectar CSS al HTML	10
5.1.	Selectores: como CSS encuentra los elementos HTML	10
6.	1: Como funciona CSS y el reset minimo	11
6.1.	Que es CSS y como se conecta al HTML	11
6.2.	El selector, la propiedad y el valor	11
6.3.	El reset: por que es obligatorio	12
7.	2: Unidades de medida	13
8.	3: Variables CSS (Custom Properties)	14
9.	4: Box Model – margin, padding, border	16
10.5:	Flexbox – distribuir en una dimension	17

11.6: CSS Grid – layout de pagina completo	19
12.7: Responsive mobile-first con media queries	20
13.8: Estilos de formulario accesibles	22
14.9: Botones con transiciones y estados	23
15.10: Modo oscuro automatico y manual	25
16.11: Animaciones y accesibilidad	27
16.1. Enlace de salto (skip link) accesible	28
17.FINAL: Mapa completo del CSS de la practica	28
 III JavaScript ES6+ – Comportamiento (Paso 4 de la practica)	 30
18.Como conectar JavaScript al HTML	30
18.1. Como JavaScript accede a los elementos HTML	31
19.1: Variables y tipos de dato	31
19.1. const y let: declarar variables	31
19.2. Template literals: texto con variables	32
19.3. Objetos y arrays	33
20.2: Funciones y arrow functions	33
20.1. Funciones classicas y arrow functions	34
20.2. Funciones que reciben funciones (callbacks)	34
21.3: El DOM (Document Object Model)	35
21.1. Seleccionar elementos del HTML	35
21.2. Modificar el contenido y atributos	35
21.3. Escuchar eventos	36
22.4: Modulos ES6 (import / export)	36
22.1. export: exponer funciones de un modulo	37
22.2. import: usar funciones de otro modulo	37
23.5: Promesas y async/await	38
23.1. Que es una Promesa	38
23.2. async/await: Promesas legibles	38
24.6: fetch – hablar con el servidor	38
24.1. GET: obtener datos	39
24.2. POST: enviar datos	40
25.7: try/catch/finally – manejar errores	41
26.8: Manipular el DOM con datos de la API	41
26.1. Renderizar datos como HTML	42

27.9: localStorage y modo oscuro	43
28.10: Formularios con JavaScript	44
29.FINAL: Todo junto – el código de la practica	45
 IV Integracion – Como trabajan juntos HTML, CSS y JS	 46
30.Flujo completo: del HTML al resultado final	46
31.Arquitectura de archivos del PFC	48
32.Ejemplo completo: boton de tema (HTML + CSS + JS)	48

HTML5 – Estructura y contenido (Paso 2 de la practica)

Que es HTML y como se estructura

Concepto: HTML

HTML (HyperText Markup Language) define la **estructura y el contenido** de una pagina web. No es un lenguaje de programacion: es un lenguaje de marcado que usa **etiquetas** (tags) para indicar al navegador que tipo de contenido es cada elemento. El navegador lee las etiquetas y renderiza el contenido visualmente.

Cada etiqueta tiene una apertura `<tag>` y un cierre `</tag>`. Algunas son autocontenidas: `<img>`, `<input>`, `<meta>`, `<br>`, `<hr>`. Las etiquetas pueden tener **atributos** que agregan informacion: `<a href="url">` donde `href` es el atributo.

1.1 Estructura minima de un documento HTML5

HTML-1 – Documento HTML5 minimo valido

Toda pagina HTML5 comienza con `<!DOCTYPE html>` que indica al navegador que use el estandar moderno. Sin esta linea, el navegador entra en “quirks mode” y se comporta de forma impredecible.

```
1      <!DOCTYPE html>
2      <html lang="es">
3      <head>
4      <!-- Metadatos: no se ven en la pagina -->
5      <meta charset="UTF-8">
6      <meta name="viewport"
7      content="width=device-width, initial-scale=1.0">
8      <meta name="description"
9      content="Descripcion breve del sitio">
10     <title>Mi Pagina | UTEQ</title>
11     <!-- Enlace al CSS externo -->
12     <link rel="stylesheet" href="css/main.css">
13     </head>
14     <body>
15     <!-- Contenido visible aqui -->
16
17     <!-- Script JS al final, con type="module" -->
18     <script type="module" src="js/app.js"></script>
19     </body>
20     </html>
```

Atributos importantes del head

- `lang="es"`: declara el idioma. Sin esto, los lectores de pantalla pronuncian el español con fonética inglesa.
- `charset="UTF-8"`: permite caracteres especiales (tildes, ñe, emojis).
- `viewport`: hace que el sitio sea responsivo en móviles. Sin esto, el móvil muestra la página en miniatura.
- `<link rel="stylesheet">`: conecta el archivo CSS externo. Es la forma correcta (no usar `<style>` inline).
- `<script type="module">`: conecta el archivo JS con soporte para `import/export`. Se pone al final del `<body>`.

Etiquetas semánticas HTML5

Concepto: Semántica

Las etiquetas semánticas describen el **proposito** del contenido, no su apariencia. Un `<header>` dice “esto es el encabezado”; un `<div>` dice “esto es un bloque generico”. Google, los lectores de pantalla y las herramientas de desarrollo usan la semántica para entender la página.

HTML-2 – Estructura semantica completa con clases CSS

Cada etiqueta semantica tiene un atributo `class` que la conecta con el CSS. El atributo `role` refuerza la semantica para lectores de pantalla. Los atributos `aria-*` agregan informacion de accesibilidad.

```
1      <body>
2      <!-- Skip link: primer elemento del body -->
3      <a href="#contenido-principal" class="saltar-nav">
4      Ir al contenido principal
5      </a>
6
7      <header role="banner">
8      <h1>Nombre del PFC</h1>
9      <nav aria-label="Navegacion principal">
10     <ul>
11     <li><a href="#inicio"
12     aria-current="page">Inicio</a></li>
13     <li><a href="#datos">Datos</a></li>
14     <li><a href="#registro">Registro</a></li>
15     </ul>
16     </nav>
17     <button id="btn-tema"
18     aria-label="Cambiar a modo oscuro"
19     aria-pressed="false">
20     Modo oscuro
21     </button>
22     </header>
23
24     <main id="contenido-principal" tabindex="-1">
25     <section aria-labelledby="titulo-datos">
26     <h2 id="titulo-datos">Datos del sistema</h2>
27     <span id="estado-carga" aria-live="polite"
28     role="status"></span>
29     <div id="contenedor-datos" role="list"
30     aria-label="Lista de elementos"></div>
31     </section>
32
33     <section aria-labelledby="titulo-form">
34     <h2 id="titulo-form">Registro</h2>
35     <form id="formulario-pfc" novalidate>
36     <!-- Campos del formulario aqui -->
37     </form>
38     </section>
39     </main>
40
41     <aside aria-label="Informacion adicional">
42     <h2>Acerca del sistema</h2>
43     <p>Descripcion del PFC.</p>
44     </aside>
45
46     <footer role="contentinfo">
47     <p>UTEQ 2025 &mdash; Ing. de Software</p>
48     </footer>
49
50     <script type="module" src="js/app.js"></script>
51     </body>
```

Donde se usa en la practica: index.html – estructura completa

Este ejemplo es exactamente el `<body>` del archivo `index.html` de la practica. Cada etiqueta semantica (`header`, `nav`, `main`, `section`, `aside`, `footer`) se conecta con el CSS via las propiedades `grid-area` del layout Grid que se define en el Paso 3.

Formularios HTML5 con validacion nativa

Concepto: Formularios accesibles

Un formulario accesible tiene 4 elementos por cada campo: (1) un `<label>` con atributo `for` igual al `id` del input, (2) un `<input>` con tipo, validacion y `aria-describedby`, (3) un `<span>` de error con `role="alert"`, y (4) opcionalmente un texto de ayuda. Los grupos de checkboxes y radios van dentro de `<fieldset>` con `<legend>`.

HTML-3 – Formulario con 8 tipos de input

Cada campo usa la clase CSS `campo-form` que en el Paso 3 se estiliza con Flexbox vertical. Los atributos `required`, `minlength`, `pattern` y `min/max` activan la validación nativa del navegador sin JavaScript.

```

1      <form id="formulario-pfc" novalidate
2      aria-describedby="instrucciones-form">
3      <p id="instrucciones-form">
4      Campos con * son obligatorios.
5      </p>
6
7      <!-- 1. text -->
8      <div class="campo-form">
9      <label for="nombre">Nombre *</label>
10     <input type="text" id="nombre" name="nombre"
11     required minlength="3"
12     placeholder="Ej: Maria Garcia"
13     aria-describedby="nombre-error">
14     <span id="nombre-error" class="mensaje-error"
15     role="alert"></span>
16     </div>
17
18     <!-- 2. email -->
19     <div class="campo-form">
20     <label for="email">Email *</label>
21     <input type="email" id="email" name="email"
22     required placeholder="user@dominio.com">
23     </div>
24
25     <!-- 3. password -->
26     <div class="campo-form">
27     <label for="clave">Contraseña * (min 8)</label>
28     <input type="password" id="clave" name="clave"
29     required minlength="8">
30     </div>
31
32     <!-- 4. number -->
33     <div class="campo-form">
34     <label for="edad">Edad *</label>
35     <input type="number" id="edad" name="edad"
36     required min="18" max="99">
37     </div>
38
39     <!-- 5. date -->
40     <div class="campo-form">
41     <label for="fecha">Fecha nacimiento *</label>
42     <input type="date" id="fecha"
43     name="fecha_nacimiento" required>
44     </div>
45
46     <!-- 6. tel -->
47     <div class="campo-form">
48     <label for="tel">Telefono</label>
49     <input type="tel" id="tel" name="telefono"
50     pattern="[0-9]{10}"
51     placeholder="0991234567">
52     </div>
53
54     <!-- 7. range -->
55     <div class="campo-form">
56     <label for="rango">Rango *</label>
57     <input type="range" id="rango" name="rango"
58     required min="0" max="100">
59     </div>
60
61     <!-- 8. checkbox -->
62     <div class="campo-form">
63     <label for="acepto">Acepto los términos y condiciones *</label>
64     <input type="checkbox" id="acepto" name="acepto"
65     required>
66     </div>
67
68     </form>

```

Como el CSS y JS se conectan con este HTML

- **CSS:** la clase `campo-form` se estiliza con `display: flex; flex-direction: column;` en el Paso 3. La clase `btn-primario` recibe los 5 estados (`normal`, `hover`, `active`, `focus-visible`, `disabled`). La clase `mensaje-error` se colorea con `var(-color-error)`.
- **JavaScript:** el `id="formulario-pfc"` se usa en el Paso 4 con `document.getElementById('formulario-pfc')` para capturar el evento `submit`. El `id="estado-carga"` se actualiza con `textContent` desde `mostrarCarga()`.
- **ARIA:** `aria-live="polite"` en el `span` de estado hace que el lector de pantalla anuncie los cambios cuando JavaScript modifica su `textContent`.

Atributos ARIA y accesibilidad

HTML-4 – Atributos ARIA mas usados en la practica

ARIA (Accessible Rich Internet Applications) agrega informacion que las etiquetas HTML no pueden expresar por si solas. Solo usar ARIA cuando no existe una etiqueta semantica equivalente.

```

1      <!-- aria-label: describe elementos sin texto visible -->
2      <button aria-label="Cerrar modal">X</button>
3
4      <!-- aria-live: anuncia cambios dinamicos -->
5      <div aria-live="polite" role="status">
6      <!-- JS cambia este texto y el lector lo anuncia -->
7      </div>
8
9      <!-- aria-pressed: estado de un boton toggle -->
10     <button aria-pressed="false">Modo oscuro</button>
11
12     <!-- aria-describedby: vincula campo con su error -->
13     <input id="email" aria-describedby="email-error">
14     <span id="email-error" role="alert"></span>
15
16     <!-- aria-current: indica la pagina activa -->
17     <a href="#inicio" aria-current="page">Inicio</a>
18
19     <!-- aria-labelledby: titulo de una seccion -->
20     <section aria-labelledby="titulo-datos">
21     <h2 id="titulo-datos">Datos</h2>
22     </section>

```

Donde se usa en la practica: index.html – todos los atributos ARIA

Cada atributo ARIA del ejemplo aparece en el HTML de la practica. En el Paso 4, JavaScript modifica `aria-pressed` del boton de tema con `btn.setAttribute('aria-pressed', true)` y actualiza el `textContent` del `span` con `aria-live` para que el lector de pantalla anuncie “Cargando datos...” o “Registro exitoso”.

CSS3 – Apariencia visual (Paso 3 de la practica)

Como conectar CSS al HTML

Concepto: 3 formas de aplicar CSS

CSS se puede aplicar de 3 formas. En la practica usamos SOLO la primera:

1. **Archivo externo** (la correcta): `<link rel="stylesheet" href="css/main.css">` en el `<head>`. Separa estructura (HTML) de presentacion (CSS). Es cacheable y reutilizable.
2. **Etiqueta style** (aceptable para prototipos): `<style>` en el `<head>`. Mezcla CSS con HTML. No se reutiliza.
3. **Inline** (nunca usar): `<p style="color:red">`. Imposible de mantener. Tiene la mayor especificidad, lo que rompe la cascada.

5.1 Selectores: como CSS encuentra los elementos HTML

CSS-0 – Selectores y como se aplican al HTML

El selector indica A QUIEN se aplica la regla. El CSS busca en el HTML los elementos que coinciden con el selector y les aplica las propiedades.

```
1      /* Selector de ETIQUETA: aplica a todos los <h1> */
2      h1 { color: #0066cc; }
3
4      /* Selector de CLASE: aplica a class="tarjeta" */
5      /* En HTML: <div class="tarjeta">...</div> */
6      .tarjeta { padding: 1.5rem; }
7
8      /* Selector de ID: aplica a id="header" (unico) */
9      /* En HTML: <header id="header">...</header> */
10     #header { background: navy; }
11
12     /* Selector de ATRIBUTO: aplica al que tenga ese attr */
13     /* En HTML: <html data-tema="oscuro"> */
14     [data-tema="oscuro"] { --color-fondo: #0f172a; }
15
16     /* Selector COMPUESTO: <a> dentro de <nav> */
17     nav a { color: white; text-decoration: none; }
18
19     /* Pseudo-clase: estado del elemento */
20     .btn:hover { background: #0055aa; } /* raton encima */
21     .btn:focus { outline: 3px solid blue; } /* con foco */
22     input:invalid { border-color: red; } /* campo invalido */
```

Regla de oro: clase para estilos, id para JavaScript

Usar `class` para aplicar CSS (un mismo estilo se aplica a multiples elementos). Usar `id` para que JavaScript encuentre un elemento especifico con `getElementById`. Evitar usar `id` para CSS porque su especificidad es demasiado alta y dificulta sobrescribir estilos.

1: Como funciona CSS y el reset minimo

6.1 Que es CSS y como se conecta al HTML

Concepto: CSS

CSS (Cascading Style Sheets) define la **apariencia visual** del HTML: colores, tamanos, posiciones, espaciado, animaciones. Se conecta al HTML de 3 formas: (1) archivo externo con `<link>` (la mejor), (2) etiqueta `<style>` en el `<head>`, (3) atributo `style=""` inline (la peor, no usar).

Ejemplo 1 – Conectar CSS externo al HTML

```
1      <!-- En el <head> del HTML: -->
2      <link rel="stylesheet" href="css/main.css">
```

6.2 El selector, la propiedad y el valor

Ejemplo 2 – Anatomia de una regla CSS

```
1      /* selector { propiedad: valor; } */
2
3      h1 {
4          color: #0066cc;          /* color del texto */
5          font-size: 2rem;         /* tamaño de la fuente */
6          margin-bottom: 1rem;    /* espacio debajo */
7      }
8
9      /* Selector de clase: aplica a todo elemento con class="tarjeta"
10         */
11     .tarjeta {
12         background: #f8f9fa;
13         padding: 1.5rem;
14     }
15
16     /* Selector de ID: aplica a UN solo elemento con id="header" */
17     #header {
18         background: #0066cc;
```

6.3 El reset: por que es obligatorio

Concepto: Reset CSS

Cada navegador tiene estilos por defecto diferentes. Chrome pone un margen de 8px al body, Firefox pone 8px pero con reglas distintas para listas y tablas, Safari tiene otra variante. Estas diferencias causan que un sitio se vea diferente en cada navegador.

El reset elimina esas diferencias estableciendo una base comun. Existen dos enfoques principales: (1) el **reset completo** (como el de Eric Meyer) que elimina absolutamente todos los estilos por defecto, y (2) el **reset minimo** (como Normalize.css) que solo corrige las inconsistencias. En la practica usamos un reset minimo de 3 reglas porque es suficiente para proyectos modernos.

El selector `*` (asterisco) selecciona todos los elementos del documento. Los pseudo-elementos `*::before` y `*::after` se incluyen porque tambien son cajas que necesitan el mismo `box-sizing`.

Ejemplo 3 – Reset minimo (el que usa la practica)

```
1      /* Aplicar a TODOS los elementos */
2      *, *::before, *::after {
3          box-sizing: border-box; /* padding y border incluidos en width
4              */
5          margin: 0;              /* quitar margenes por defecto */
6          padding: 0;             /* quitar padding por defecto */
7      }
8
9      /* Imagenes y videos: nunca desbordar su contenedor */
10     img, video {
11         max-width: 100%;
12         display: block;
13     }
```

Concepto: box-sizing: border-box

Sin `border-box`: un elemento con `width: 200px` y `padding: 20px` mide 240px ($200 + 20 + 20$). Con `border-box`: mide 200px (el padding se resta del width). Sin esto, los layouts se rompen constantemente.

Donde se usa en la practica: main.css – primeras lineas

Estas 3 reglas son las primeras lineas del archivo `main.css` de la practica. Sin ellas, todo el layout posterior se comporta de forma impredecible.

2: Unidades de medida

Concepto: px vs rem vs em vs % vs vw

CSS tiene unidades absolutas y relativas. Entender la diferencia es fundamental para crear sitios que se adapten a las preferencias del usuario:

- **px** (pixel): unidad absoluta. $1\text{px} = 1$ punto en la pantalla. NO escala si el usuario cambia el tamaño de fuente en su navegador. Usar solo para bordes (`border: 1px`).
- **rem** (root em): relativa al `font-size` del elemento `<html>` (por defecto 16px). $1\text{rem} = 16\text{px}$, $1.5\text{rem} = 24\text{px}$. SI escala con la preferencia del usuario. Es la unidad recomendada para tipografía, padding y margin.
- **em**: relativa al `font-size` del elemento padre (no del root). Si un `<div>` tiene `font-size: 20px`, entonces $1\text{em} = 20\text{px}$ dentro de ese div. Es útil para componentes que deben escalar proporcionalmente, pero puede causar confusión cuando se anidan (“efecto cascada”).
- **%**: relativa al elemento padre. `width: 80%` = 80% del ancho del padre. Muy usada para layouts fluidos.
- **vw / vh**: relativa al viewport (ventana del navegador). $1\text{vw} = 1\%$ del ancho de la ventana, $1\text{vh} = 1\%$ del alto. Usada en tipografía fluida con `clamp()`.

Regla practica: usar **rem** para texto y espaciado, **%** para anchos de contenedores, **px** solo para bordes, y **vw** dentro de `clamp()` para tipografía fluida.

Ejemplo 4 – Comparacion de unidades

```

1      /* px: absoluto. 1px = 1 pixel. NO escala si el usuario
2      cambia el tamaño de fuente en su navegador. */
3      h1 { font-size: 32px; }
4
5      /* rem: relativo al font-size del <html> (por defecto 16px).
6      1rem = 16px. SI escala con la preferencia del usuario.
7      SIEMPRE preferir rem sobre px para texto y espaciado. */
8      h1 { font-size: 2rem; } /* 2 * 16px = 32px */
9
10     /* %: relativo al elemento padre.
11     width: 80% = 80% del ancho del padre. */
12     .contenedor { width: 80%; }
13
14     /* vw: relativo al ancho del viewport (ventana).
15     1vw = 1% del ancho de la ventana.
16     Util en tipografía fluida con clamp(). */
17     h1 { font-size: 4vw; } /* 4% del ancho de la ventana */
18
19     /* clamp(minimo, preferido, maximo):
20     Nunca menor de 1.75rem, idealmente 4vw,
21     nunca mayor de 2.5rem. */
22     h1 { font-size: clamp(1.75rem, 4vw, 2.5rem); }
```

Donde se usa en la practica: variables.css

Todas las variables de tipografia en la practica usan `clamp()` para que el texto sea fluido sin media queries: `-tamano-base: clamp(1rem, 1.5vw + 0.5rem, 1.125rem);`

3: Variables CSS (Custom Properties)

Concepto: Variables CSS (Custom Properties)

Las variables CSS se declaran en `:root` con `-nombre: valor;` y se usan con `var(-nombre)`. Permiten cambiar toda la paleta de colores del sitio desde un solo lugar. Son la base del modo oscuro.

Por que usar variables en lugar de valores directos:

- **Mantenibilidad:** si el cliente pide cambiar el azul corporativo, se modifica UNA linea en `:root` en lugar de buscar y reemplazar en 50 lugares.
- **Consistencia:** todos los componentes usan exactamente el mismo azul, el mismo espaciado, el mismo radio de borde. No hay variaciones accidentales.
- **Modo oscuro:** al cambiar las variables dentro de una media query o un selector como `[data-tema="oscuro"]`, TODOS los elementos que usan esas variables cambian automaticamente sin escribir reglas adicionales.
- **Lectura:** `color: var(-color-primario)` es mas descriptivo que `color: #0066cc`. El nombre de la variable explica su proposito.

El selector `:root` equivale al elemento `<html>` y garantiza que las variables esten disponibles en todo el documento. Se recomienda organizar las variables por categoria: colores, tipografia, espaciado, layout y transiciones.

Ejemplo 5 – Declarar y usar variables

```
1      /* Declarar en :root (accesibles en todo el documento) */
2      :root {
3          --color-primario: #0066cc;
4          --color-fondo:    #ffffff;
5          --color-texto:    #1a1a1a;
6          --esp-md:         1rem;
7          --radio-borde:    6px;
8      }
9
10     /* Usar con var() */
11     body {
12         background-color: var(--color-fondo);
13         color: var(--color-texto);
14     }
15
16     .btn {
17         background: var(--color-primario);
18         padding: var(--esp-md);
19         border-radius: var(--radio-borde);
20     }
```

Resultado: poder de las variables CSS

Si cambias `--color-primario: #e74c3c;` en `:root`, TODOS los elementos que usan `var(--color-primario)` cambian automaticamente: el boton, el header, el focus de los inputs, el color de los enlaces.

Prueba practica: abrir el archivo `variables.css`, cambiar el valor de `--color-primario` de `#0066cc` (azul) a `#e74c3c` (rojo), guardar, y observar cuantos elementos cambian en el navegador sin tocar ninguna otra linea de CSS.

Esto demuestra el principio DRY (Don't Repeat Yourself): cada valor se define una sola vez en un solo lugar. Si el cliente pide cambiar el color corporativo, se modifica una linea en lugar de buscar y reemplazar en decenas de archivos. Las variables CSS tambien se pueden leer y modificar desde JavaScript con `document.documentElement.style.setProperty('--color-primario', '#e74c3c')`, lo que permite al usuario personalizar los colores del sitio en tiempo real.

Ejemplo 6 – Escala de espaciado consistente

```

1      :root {
2          /* Escala de 4px (cada valor es el doble del anterior) */
3          --esp-xs: 0.25rem;    /* 4px */
4          --esp-sm: 0.5rem;     /* 8px */
5          --esp-md: 1rem;       /* 16px */
6          --esp-lg: 1.5rem;     /* 24px */
7          --esp-xl: 2rem;       /* 32px */
8      }
9
10     /* Usar consistentemente */
11     .campo-form { margin-bottom: var(--esp-md); }
12     .tarjeta     { padding: var(--esp-lg); }
13     header      { padding: var(--esp-md) var(--esp-lg); }

```

Donde se usa en la practica: variables.css – archivo completo

El archivo `variables.css` de la practica declara 22 variables organizadas en 5 categorias: colores principales y de superficies (6 variables de color como `-color-primario`, `-color-fondo`, `-color-texto`), tipografia (4 variables con `clamp()` para tamanos fluidos y `-fuente-base` con la pila de fuentes del sistema), espaciado (6 variables en escala de 4px: `-esp-xs` hasta `-esp-2xl`), layout (4 variables como `-ancho-max` y `-sombra-sm`) y transiciones (2 variables para duraciones estandar). El archivo `main.css` las consume con `var()` en cada propiedad, de modo que ningun valor numerico aparece repetido fuera de `:root`.

4: Box Model – margin, padding, border**Concepto: El modelo de caja**

Todo elemento HTML es una caja con 4 capas, de adentro hacia afuera:

1. **Content:** el contenido real del elemento (texto, imagen, video). Su tamano se controla con `width` y `height`.
2. **Padding:** espacio transparente entre el contenido y el borde. Empuja el contenido hacia adentro. Se define con `padding` (1-4 valores en orden del reloj: top, right, bottom, left).
3. **Border:** la linea visible alrededor del elemento. Se define con `border: ancho estilo color` (ej: `border: 2px solid #dee2e6`). `border-radius` redondea las esquinas.
4. **Margin:** espacio transparente entre este elemento y los elementos vecinos. Empuja otros elementos hacia afuera. `margin: 0 auto` centra horizontalmente. Los margenes verticales entre elementos adyacentes colapsan: si un elemento tiene `margin-bottom: 20px` y el siguiente tiene `margin-top: 30px`, el espacio real entre ellos es 30px (el mayor), no 50px.

Propiedad adicional: `box-shadow` agrega una sombra visual que no afecta el flujo del documento (no ocupa espacio). Su sintaxis es: `box-shadow: desplazamiento-X desplazamiento-Y desenfoque color;`. Se puede agregar `inset` para sombras internas.

Ejemplo 7 – Padding, margin y border

```
1  .tarjeta {
2    /* Padding: espacio INTERIOR (entre el texto y el borde) */
3    padding: 1.5rem;
4
5    /* Border: linea visible alrededor */
6    border: 1px solid #dee2e6;
7
8    /* Border-radius: esquinas redondeadas */
9    border-radius: 12px;
10
11   /* Margin: espacio EXTERIOR (entre esta caja y las demas) */
12   margin-bottom: 1rem;
13
14   /* Box-shadow: sombra (no afecta el flujo del documento) */
15   box-shadow: 0 2px 8px rgba(0, 0, 0, 0.12);
16   /* Desglose: desplazamiento-X desplazamiento-Y desenfoque
17   color */
18 }
```

Ejemplo 8 – Shorthand de margin y padding (orden del reloj)

```
1  /* 4 valores: top right bottom left (sentido del reloj) */
2  padding: 10px 20px 10px 20px;
3
4  /* 2 valores: vertical horizontal */
5  padding: 10px 20px; /* arriba/abajo=10px, lados=20px */
6
7  /* 1 valor: los 4 iguales */
8  padding: 1rem;
9
10 /* Centrar un bloque horizontalmente */
11 margin: 0 auto; /* 0 arriba/abajo, auto a los lados */
```

Donde se usa en la practica: main.css – tarjetas y formulario

Las tarjetas usan `padding: var(-esp-lg)`, `border: 1px solid var(-color-borde)`, `border-radius: var(-radio-borde-lg)` y `box-shadow: var(-sombra-sm)`.

5: Flexbox – distribuir en una dimension

Concepto: Flexbox

Flexbox distribuye elementos en **una dimension**: una fila O una columna. Se activa con `display: flex` en el contenedor padre. Los hijos se distribuyen automaticamente.

Las propiedades mas importantes del contenedor flex son: `flex-direction` (fila o columna), `justify-content` (distribucion en el eje principal), `align-items` (alineacion en el eje secundario), `flex-wrap` (permitir que los hijos salten de linea) y `gap` (espacio entre hijos).

Para los hijos, la propiedad `flex` combina tres valores: cuanto crece (`flex-grow`), cuanto se encoge (`flex-shrink`) y su tamaño base (`flex-basis`).

Ejemplo 9 – Navbar horizontal con Flexbox

El header necesita dos grupos de contenido en los extremos opuestos: el título a la izquierda y la navegación a la derecha. `space-between` los separa automáticamente. La lista de enlaces dentro del nav también es flex para distribuirlos horizontalmente con espacio uniforme.

```
1      /* Contenedor flex: distribuye hijos en fila */
2      header {
3          display: flex;
4          align-items: center;          /* centrar verticalmente */
5          justify-content: space-between; /* título a la izq, nav a la der
6                                          */
7          gap: 1rem;                   /* espacio entre hijos */
8          padding: 1rem 1.5rem;
9      }
10
11     /* La lista de navegación también es flex */
12     nav ul {
13         display: flex;
14         gap: 1rem;          /* espacio entre cada enlace */
15         list-style: none; /* quitar los puntos de la lista */
16     }
```

Ejemplo 10 – Tarjetas con flex-wrap (se ajustan al espacio)

Las tarjetas de datos del PFC deben adaptarse al espacio disponible: 3 por fila en desktop, 2 en tablet, 1 en mobile. Con `flex-wrap: wrap` y un `flex-basis` mínimo, Flexbox lo resuelve sin media queries.

```
1      .tarjetas-grid {
2          display: flex;
3          flex-wrap: wrap; /* si no caben en una fila, saltan a la
4                          siguiente */
5          gap: 1rem;
6      }
7
8      .tarjeta {
9          flex: 1 1 260px;
10         /* Desglose:
11         flex-grow: 1      = CRECE para llenar el espacio disponible
12         flex-shrink: 1    = SE ENCOGE si el espacio es insuficiente
13         flex-basis: 260px = tamaño base mínimo de 260px
14         Resultado: las tarjetas miden al menos 260px y crecen
15         para llenar la fila. Si no caben 2, se apilan. */
16     }
```

Ejemplo 11 – Formulario vertical con Flexbox

Cada campo del formulario tiene 3 elementos apilados verticalmente: el label arriba, el input en el medio y el mensaje de error abajo. `flex-direction: column` los organiza de arriba a abajo con un `gap` pequeño entre ellos.

```
1 .campo-form {  
2   display: flex;  
3   flex-direction: column; /* apilar label, input y error vertical  
   */  
4   gap: 0.25rem;          /* pequeno espacio entre cada uno */  
5   margin-bottom: 1rem;  
6 }
```

Donde se usa en la practica: main.css – header, tarjetas y formulario

La practica usa Flexbox en 3 lugares distintos, cada uno con una configuracion diferente:

- **Header:** `display: flex` con `justify-content: space-between` para poner el titulo a la izquierda y la navegacion a la derecha. `align-items: center` centra verticalmente ambos. `flex-wrap: wrap` permite que se apilen en pantallas pequenas.
- **Tarjetas:** `display: flex` con `flex-wrap: wrap` y `gap: 1rem`. Cada tarjeta tiene `flex: 1 1 260px`: crece para llenar el espacio, se encoge si falta, pero nunca mide menos de 260px. En pantalla ancha caben 3 tarjetas por fila; en mobile se apilan a 1.
- **Formulario:** cada `.campo-form` usa `flex-direction: column` para apilar verticalmente el label, el input y el mensaje de error, con `gap: 0.25rem` entre cada uno.

La diferencia clave con Grid es que Flexbox no controla las filas: si caben 3 tarjetas caben, si caben 2 se ajustan, si cabe 1 se apila. Grid forzaria exactamente 3 columnas.

6: CSS Grid – layout de pagina completo

Concepto: CSS Grid

Grid distribuye elementos en **dos dimensiones**: filas Y columnas al mismo tiempo. Se usa para el layout general de la pagina (header, main, aside, footer). Se activa con `display: grid` en el contenedor padre.

La propiedad `grid-template-areas` permite dibujar el layout visualmente en el codigo usando nombres de area entre comillas. Cada fila es un string y cada columna es un nombre separado por espacios. Luego, cada elemento HTML se asigna a su area con `grid-area: nombre;`. Esta sintaxis es la mas legible y mantenible para layouts de pagina.

Ejemplo 12 – Grid basico con areas nombradas

Usa `grid-template-areas` para nombrar cada zona del layout. Al cambiar las media queries, se reorganiza el layout modificando solo las areas, sin tocar el HTML.

```

1      .layout {
2          display: grid;
3          grid-template-areas:
4              "header header"      /* header ocupa las 2 columnas */
5              "main  aside"        /* main a la izquierda, aside a la derecha
6              */
7              "footer footer";     /* footer ocupa las 2 columnas */
8          grid-template-columns: 1fr 280px; /* main=flexible, aside=280px
9              */
10         gap: 1rem;
11         min-height: 100vh; /* minimo el alto de la ventana */
12     }
13
14     /* Asignar cada elemento a su area */
15     header { grid-area: header; }
16     main    { grid-area: main; }
17     aside   { grid-area: aside; }
18     footer  { grid-area: footer; }

```

Concepto: fr (fraction unit)

`1fr` significa “una fraccion del espacio disponible”. En `grid-template-columns: 1fr 280px`, la primera columna toma todo el espacio que sobra despues de asignar 280px al aside. Si el contenedor mide 1000px: $\text{main} = 1000 - 280 - \text{gap} = 704\text{px}$.

Se pueden combinar multiples `fr`: `grid-template-columns: 2fr 1fr` crea dos columnas donde la primera es el doble de ancha que la segunda. La funcion `repeat()` simplifica la repeticion: `repeat(3, 1fr)` equivale a `1fr 1fr 1fr` (3 columnas iguales).

Donde se usa en la practica: main.css – layout principal

Este ejemplo es exactamente el layout de la practica en la version tablet/desktop.

7: Responsive mobile-first con media queries**Concepto: Mobile-first**

Se escribe primero el CSS para la pantalla mas pequena (mobile). Luego se agregan media queries con `min-width` para pantallas mas grandes. Esto es mas eficiente porque el CSS base es el mas simple y las media queries solo agregan complejidad cuando hay espacio.

Por que mobile-first y no desktop-first:

- En el enfoque **desktop-first**, el CSS base es complejo (2 columnas, margenes amplios) y las media queries con `max-width` van quitando cosas para pantallas pequenas. Esto genera mas codigo y es mas propenso a errores.

- En el enfoque **mobile-first**, el CSS base es simple (1 columna, sin márgenes laterales) y las media queries con `min-width` agregan complejidad solo cuando hay espacio para ello. Menos código, menos bugs.
- Google indexa primero la versión mobile del sitio (**mobile-first indexing**), así que el CSS base debe ser el mobile.

Breakpoints mas usados: 768px (tablet), 1024px (desktop), 1200px (pantalla ancha). En la practica usamos los primeros dos.

Ejemplo 13a – Mobile (base, sin media query)

```
1      /* MOBILE: 1 columna (CSS base, sin media query) */
2      .layout {
3          display: grid;
4          grid-template-areas:
5              "header"
6              "main"
7              "aside"
8              "footer";
9          gap: 1rem;
10         padding: 1rem;
11     }
```

Ejemplo 13b – Tablet y Desktop (media queries)

```
1      /* TABLET (768px+): 2 columnas */
2      @media (min-width: 768px) {
3          .layout {
4              grid-template-areas:
5                  "header header"
6                  "main  aside"
7                  "footer footer";
8              grid-template-columns: 1fr 280px;
9              padding: 1.5rem;
10         }
11     }
12
13     /* DESKTOP (1024px+): centrado */
14     @media (min-width: 1024px) {
15         .layout {
16             grid-template-columns: 1fr 320px;
17             max-width: 1200px;
18             margin: 0 auto;
19             padding: 2rem;
20         }
21     }
```

Como verificar el responsive

F12 en Chrome → icono de dispositivo movil (Toggle device toolbar) → cambiar el ancho: 375px (mobile, 1 columna) → 768px (tablet, 2 columnas) → 1024px (desktop, centrado). El aside pasa de estar debajo del main a estar al lado.

Donde se usa en la practica: main.css – el layout completo

Este ejemplo es exactamente el layout responsive de la practica con las mismas 3 etapas y los mismos breakpoints.

8: Estilos de formulario accesibles

Ejemplo 14a – Base de todos los inputs

```
1  input, select, textarea {
2      padding: 0.5rem 1rem;
3      border: 2px solid #dee2e6;
4      border-radius: 6px;
5      font-size: 1rem;
6      font-family: inherit; /* hereda la fuente del body */
7      background: var(--color-fondo);
8      color: var(--color-texto);
9      transition: border-color 150ms ease;
10 }
```

Ejemplo 14b – Focus, validacion y mensajes de error

```
1  /* Focus: anillo azul al hacer clic o Tab */
2  input:focus, select:focus, textarea:focus {
3      outline: 3px solid #0066cc;
4      outline-offset: 2px;
5      border-color: #0066cc;
6  }
7
8  /* Invalido SOLO si el usuario ya escribio algo */
9  input:invalid:not(:placeholder-shown) {
10     border-color: #dc3545; /* borde rojo */
11 }
12
13 /* Textos de ayuda y error */
14 .mensaje-error { color: #dc3545; font-size: 0.875rem; }
15 .texto-ayuda   { color: #555; font-size: 0.875rem; }
```

Concepto: :focus-visible vs :focus

:focus se activa siempre que el elemento recibe foco (teclado O raton). :focus-visible se activa solo cuando el foco es por teclado (Tab). Los botones usan :focus-visible para no mostrar el outline cuando el usuario hace clic con el raton (que es molesto) pero si cuando navega con Tab (que es necesario para accesibilidad).

Los inputs de formulario usan `:focus` (no `:focus-visible`) porque el outline debe aparecer tanto al hacer clic como al navegar con Tab, ya que el usuario necesita ver cual campo esta editando en ambos casos. Las mejores practicas de accesibilidad exigen que el **outline** sea de al menos 3px y tenga un **outline-offset** de 2-3px para que no se confunda con el borde del elemento.

Donde se usa en la practica: main.css – formulario

La practica usa exactamente estos estilos. El selector `:invalid:not(:placeholder-shown)` evita que el campo se muestre rojo antes de que el usuario escriba (porque un campo vacio es invalido si tiene **required**, pero no queremos asustarlo antes de que empiece).

9: Botones con transiciones y estados

Ejemplo 15a – Boton: estilo base

```
1      .btn-primario {  
2          background: var(--color-primario);  
3          color: #fff;  
4          padding: 0.5rem 2rem;  
5          border: none;  
6          border-radius: 6px;  
7          font-size: 1rem;  
8          font-family: inherit;  
9          cursor: pointer;  
10         transition: background 150ms ease, transform 150ms ease;  
11     }
```

Ejemplo 15b – Boton: hover, active, focus-visible y disabled

```

1      /* Hover: raton encima. Oscurecer el fondo. */
2      .btn-primario:hover {
3          background: var(--color-primario-hover);
4      }
5
6      /* Active: mientras se mantiene presionado */
7      .btn-primario:active {
8          transform: scale(0.98); /* se encoge 2% */
9      }
10
11     /* Focus-visible: solo con teclado (Tab) */
12     .btn-primario:focus-visible {
13         outline: 3px solid var(--color-primario);
14         outline-offset: 3px;
15     }
16
17     /* Disabled: boton deshabilitado */
18     .btn-primario:disabled {
19         opacity: 0.5;           /* semitransparente */
20         cursor: not-allowed;    /* icono de prohibido */
21         pointer-events: none;   /* ignora clics */
22     }

```

Concepto: transition

La propiedad **transition** anima los cambios de valor de otras propiedades CSS. Su sintaxis es:

transition: propiedad duracion curva retardo;

- **propiedad:** cual propiedad animar (background, color, transform, all).
- **duracion:** cuanto dura la animacion (150ms, 0.3s). Regla: 100-300ms para hover de botones/enlaces, 300-500ms para cambios de layout o tema.
- **curva:** la aceleracion de la animacion. **ease** (lento-rapido-lento, la mas natural), **linear** (velocidad constante), **ease-in** (empieza lento), **ease-out** (termina lento), **ease-in-out** (lento en ambos extremos).
- **retardo** (opcional): esperar antes de iniciar (50ms, 0.1s).

Se pueden animar multiples propiedades separandolas con coma: **transition: background 150ms ease, transform 150ms ease;**. Usar **all** anima todas las propiedades que cambien, pero es menos eficiente porque el navegador debe vigilar todas.

Propiedades que se pueden animar: las que tienen valores intermedios calculables: colores (color, background), tamanos (width, height, padding), posiciones (top, left), transformaciones (transform), opacidad (opacity). **No se pueden animar:** display, font-family, position.

10: Modo oscuro automatico y manual

Ejemplo 16 – Modo oscuro con media query del sistema

```
1      /* === MODO CLARO (base, siempre se carga primero) === */
2      :root {
3          --color-fondo:          #ffffff;    /* blanco */
4          --color-fondo-tarjeta: #f8f9fa;    /* gris muy claro */
5          --color-texto:          #1a1a1a;    /* casi negro */
6          --color-texto-suave:    #555555;    /* gris medio */
7          --color-borde:          #dee2e6;    /* gris claro */
8      }
9
10     /* === MODO OSCURO AUTOMATICO ===
11     Esta media query detecta si el sistema operativo
12     del usuario tiene configurado el tema oscuro.
13     Solo redefine las variables de color; el resto
14     del CSS no cambia ni una linea. */
15     @media (prefers-color-scheme: dark) {
16         :root {
17             --color-fondo:          #0f172a;    /* azul muy oscuro */
18             --color-fondo-tarjeta: #1e293b;    /* azul oscuro */
19             --color-texto:          #f1f5f9;    /* casi blanco */
20             --color-texto-suave:    #94a3b8;    /* gris azulado */
21             --color-borde:          #334155;    /* gris oscuro */
22         }
23     }
```

Ejemplo 17 – Modo oscuro manual con boton (atributo data-tema)

```

1      /* === MODO OSCURO MANUAL ===
2      Cuando el usuario presiona el boton de tema,
3      JavaScript ejecuta:
4      document.documentElement
5      .setAttribute('data-tema', 'oscuro');
6      Esto agrega data-tema="oscuro" al <html> */
7      [data-tema="oscuro"] {
8          --color-fondo:          #0f172a;
9          --color-fondo-tarjeta: #1e293b;
10         --color-texto:          #f1f5f9;
11         --color-texto-suave:    #94a3b8;
12         --color-borde:         #334155;
13     }
14
15     /* === COMBINACION: auto + manual ===
16     Si el SO tiene modo oscuro, activar el tema
17     oscuro EXCEPTO cuando el usuario eligio
18     explicitamente el modo claro con el boton.
19     :not([data-tema="claro"]) excluye ese caso. */
20     @media (prefers-color-scheme: dark) {
21         :root:not([data-tema="claro"]) {
22             --color-fondo:          #0f172a;
23             --color-fondo-tarjeta: #1e293b;
24             --color-texto:          #f1f5f9;
25             --color-texto-suave:    #94a3b8;
26             --color-borde:         #334155;
27         }
28     }

```

Ejemplo 18 – Transicion suave entre modos

```

1      body {
2          background-color: var(--color-fondo);
3          color: var(--color-texto);
4          /* Transicion: el cambio de color es gradual, no brusco */
5          transition: background-color 300ms ease,
6                     color 300ms ease;
7      }

```

Donde se usa en la practica: variables.css – modo oscuro completo

La practica combina las tres tecnicas: variables en `:root` (claro), media query para auto-deteccion del SO, y selector `[data-tema="oscuro"]` para el boton. La transicion en el body hace que el cambio sea suave.

11: Animaciones y accesibilidad

Ejemplo 19a – Definir la animacion @keyframes

```
1      @keyframes aparecer {
2          from {
3              opacity: 0;                /* invisible */
4              transform: translateY(12px); /* 12px abajo */
5          }
6          to {
7              opacity: 1;                /* visible */
8              transform: translateY(0);   /* posicion normal */
9          }
10     }
```

Ejemplo 19b – Aplicar y desactivar por accesibilidad

```
1      .tarjeta {
2          animation: aparecer 0.4s ease both;
3          /* nombre duracion curva fill-mode */
4      }
5
6      /* OBLIGATORIO: desactivar para usuarios con sensibilidad */
7      @media (prefers-reduced-motion: reduce) {
8          .tarjeta { animation: none; }
9      }
```

Concepto: prefers-reduced-motion

Hay usuarios que experimentan mareo, nauseas o convulsiones con animaciones. Sus sistemas operativos tienen una opcion para “Reducir movimiento”. La media query **prefers-reduced-motion: reduce** detecta esa preferencia. Es **obligatorio** desactivar todas las animaciones cuando esta activa. Lighthouse lo verifica.

16.1 Enlace de salto (skip link) accesible

Ejemplo 20 – Skip link oculto que aparece con Tab

```

1      /* Oculto por defecto (fuera de la pantalla) */
2      .saltar-nav {
3          position: absolute;
4          top: -100%;          /* fuera de la vista */
5          left: 0;
6          padding: 0.5rem 1rem;
7          background: #0066cc;
8          color: #fff;
9          text-decoration: none;
10         z-index: 9999;        /* siempre encima de todo */
11         transition: top 150ms ease;
12     }
13
14     /* Aparece cuando el usuario presiona Tab */
15     .saltar-nav:focus {
16         top: 0; /* entra a la pantalla */
17     }

```

Donde se usa en la practica: main.css – accesibilidad

El skip link es el primer elemento del <body> en el HTML: Ir al contenido principal. Es invisible hasta que el usuario presiona Tab, momento en que aparece en la esquina superior izquierda.

FINAL: Mapa completo del CSS de la practica

Verificacion

Si entendiste los 20 ejemplos anteriores, puedes leer y escribir cada linea del Paso 3 de la practica. La Tabla 1 mapea cada concepto a su ubicacion exacta en el codigo de la practica y al ejemplo correspondiente en este documento.

Cuadro 1: Mapa completo: concepto CSS → ubicacion en la practica → ejemplo en este documento.

#	Concepto CSS	Donde se usa en la practica	Ejemplo en este documento
1	Reset con box-sizing	Primeras lineas de main.css	Ejemplo 3
2	Unidades rem, em, %, vw, clamp()	Variables de tipografia en variables.css	Ejemplo 4
3	Variables CSS en :root	Archivo variables.css completo	Ejemplos 5 y 6
4	Padding, border, border-radius, box-shadow	Tarjetas, formulario, header	Ejemplos 7 y 8
5	Flexbox horizontal	Header: titulo + nav con space-between	Ejemplo 9

#	Concepto CSS	Donde se usa en la practica	Ejemplo en este documento
6	Flexbox con flex-wrap	Tarjetas: flex: 1 1 260px	Ejemplo 10
7	Flexbox vertical	Campos del formulario: flex-direction: column	Ejemplo 11
8	CSS Grid con areas nombradas	Layout principal: header, main, aside, footer	Ejemplo 12
9	Media queries mobile-first	3 breakpoints: base, 768px, 1024px	Ejemplo 13a y 13b
10	Focus, :invalid, :focus-visible	Inputs con outline y validacion visual	Ejemplo 14a y 14b
11	Boton con 5 estados	.btn-primario: normal, hover, active, focus, disabled	Ejemplo 15a y 15b
12	Modo oscuro automatico	@media (prefers-color-scheme: dark) en variables.css	Ejemplo 16
13	Modo oscuro manual	[data-tema="oscuro"] + :not() en variables.css	Ejemplo 17
14	Transicion suave de tema	body: transition background-color 300ms en main.css	Ejemplo 18
15	@keyframes y animation	Animacion de entrada de tarjetas en main.css	Ejemplo 19a
16	prefers-reduced-motion	Desactivar animaciones para accesibilidad	Ejemplo 19b
17	Skip link accesible	Enlace oculto que aparece con Tab en main.css	Ejemplo 20

Estas listo

Si puedes explicar cada fila de esta tabla, puedes implementar el Paso 3 completo de la practica. Abre la guia de la PE U1, ve al Paso 3, y veras que cada bloque de CSS corresponde a uno de estos 20 ejemplos.

JavaScript ES6+ – Comportamiento (Paso 4 de la practica)

Como conectar JavaScript al HTML

Concepto: script type="module"

JavaScript se conecta al HTML con la etiqueta `<script>` al final del `<body>`. El atributo `type="module"` activa el sistema de modulos ES6 (`import/export`) y hace que el script se ejecute automaticamente despues de que el DOM esta listo (equivalente a `defer`).

JS-0 – Conectar JavaScript al HTML

El script se pone al final del body para que el HTML ya exista cuando JavaScript se ejecuta. Con `type="module"`, cada archivo JS tiene su propio ambito: las variables no contaminan el espacio global.

```
1      <!-- En index.html, al final del <body>: -->
2
3      <!-- SIN modulos (viejo, no usar): -->
4      <script src="js/app.js"></script>
5
6      <!-- CON modulos (correcto para la practica): -->
7      <script type="module" src="js/app.js"></script>
8
9      <!-- La diferencia:
10     - Sin module: no se puede usar import/export
11     - Con module: import/export funcionan,
12     el script es defer automaticamente,
13     cada archivo tiene su propio ambito -->
```

18.1 Como JavaScript accede a los elementos HTML

JS-0b – JavaScript lee y modifica el HTML

JavaScript usa los `id` y `class` del HTML para encontrar elementos. Luego puede leer su contenido, cambiar sus atributos, o escuchar eventos como clics.

```
1 // Buscar un elemento por su id del HTML
2 const btn = document.getElementById('btn-tema');
3 // <button id="btn-tema"> en el HTML
4
5 // Cambiar su texto
6 btn.textContent = 'Modo claro';
7
8 // Cambiar un atributo ARIA
9 btn.setAttribute('aria-pressed', 'true');
10
11 // Escuchar cuando el usuario hace clic
12 btn.addEventListener('click', () => {
13   console.log('Boton presionado!');
14 });
15
16 // Buscar por selector CSS (como en CSS)
17 const nav = document.querySelector('nav');
18 const links = document.querySelectorAll('nav a');
19
20 // Buscar el formulario y escuchar el envio
21 const form = document.getElementById('formulario-pfc');
22 form.addEventListener('submit', (e) => {
23   e.preventDefault(); // NO recargar la pagina
24   const datos = Object.fromEntries(new FormData(form));
25   console.log(datos); // {nombre:"Ana", email:"a@b.c"}
26 });
```

Donde se usa en la practica: app.js – punto de entrada

Este ejemplo muestra exactamente como `app.js` accede a los elementos HTML creados en el Paso 2: el boton de tema por `id="btn-tema"`, el formulario por `id="formulario-pfc"`, y el contenedor de datos por `id="contenedor-datos"`.

1: Variables y tipos de dato

19.1 `const` y `let`: declarar variables

Concepto

`const` declara una variable cuya referencia no se puede reasignar. `let` permite reasignar. `var` es obsoleto: no usarlo. Regla: usar siempre `const`; solo `let` si necesitas cambiar el valor despues.

Ejemplo 1 – const vs let

```
1 // const: no se puede reasignar
2 const nombre = "Maria";
3 const edad = 22;
4 const esEstudiante = true;
5
6 // let: si se puede reasignar
7 let contador = 0;
8 contador = 1; // OK
9 contador = 2; // OK
10
11 // Esto daría ERROR:
12 // nombre = "Juan"; // TypeError: Assignment to constant variable
13
14 console.log(nombre, edad, esEstudiante, contador);
15 // Maria 22 true 2
```

Resultado en consola

Maria 22 true 2

19.2 Template literals: texto con variables**Ejemplo 2 – Template literals con backticks**

```
1 const nombre = "Carlos";
2 const edad = 20;
3
4 // ANTES (concatenacion fea):
5 const msgViejo = "Hola, " + nombre + ". Tienes " + edad + " anos."
6 ;
7
8 // AHORA (template literal con backticks):
9 const msg = `Hola, ${nombre}. Tienes ${edad} anos.`;
10
11 // Se puede poner cualquier expresion dentro de ${}:
12 const msg2 = `En 5 anos tendras ${edad + 5} anos.`;
13
14 console.log(msg); // Hola, Carlos. Tienes 20 anos.
15 console.log(msg2); // En 5 anos tendras 25 anos.
```

Donde se usa en la practica: app.js, ui.js

Los template literals se usan en `renderizarTarjetas()` para construir el HTML de cada tarjeta, y en los mensajes de error.

19.3 Objetos y arrays

Ejemplo 3 – Objetos y arrays basicos

```
1 // Objeto: coleccion de propiedades clave:valor
2 const usuario = {
3   nombre: "Ana",
4   email: "ana@uteq.edu.ec",
5   edad: 21
6 };
7
8 // Acceder a propiedades
9 console.log(usuario.nombre); // Ana
10 console.log(usuario.email); // ana@uteq.edu.ec
11
12 // Array: lista ordenada de elementos
13 const colores = ["rojo", "verde", "azul"];
14 console.log(colores[0]); // rojo
15 console.log(colores.length); // 3
16
17 // Array de objetos (lo que devuelve una API):
18 const usuarios = [
19   { id: 1, name: "Leanne", email: "l@mail.com" },
20   { id: 2, name: "Ervin", email: "e@mail.com" },
21 ];
22 console.log(usuarios[0].name); // Leanne
```

Donde se usa en la practica: api.js

La API devuelve un array de objetos exactamente como `usuarios` del ejemplo. Cada objeto tiene propiedades como `id`, `name`, `email`.

2: Funciones y arrow functions

20.1 Funciones classicas y arrow functions

Ejemplo 4 – Tres formas de escribir una funcion

```
1 // Forma 1: funcion clasica (funciona, pero es verbosa)
2 function sumar(a, b) {
3   return a + b;
4 }
5
6 // Forma 2: arrow function con cuerpo
7 const restar = (a, b) => {
8   return a - b;
9 };
10
11 // Forma 3: arrow function corta (1 expresion = return implicito)
12 const multiplicar = (a, b) => a * b;
13
14 console.log(sumar(3, 5)); // 8
15 console.log(restar(10, 4)); // 6
16 console.log(multiplicar(3, 7)); // 21
```

20.2 Funciones que reciben funciones (callbacks)

Ejemplo 5 – forEach, map y filter con arrow functions

```
1 const numeros = [1, 2, 3, 4, 5];
2
3 // forEach: ejecuta algo por cada elemento (no devuelve nada)
4 numeros.forEach(n => console.log(n * 2));
5 // 2, 4, 6, 8, 10
6
7 // map: transforma cada elemento y devuelve un nuevo array
8 const dobles = numeros.map(n => n * 2);
9 console.log(dobles); // [2, 4, 6, 8, 10]
10
11 // filter: filtra y devuelve solo los que cumplen la condicion
12 const pares = numeros.filter(n => n % 2 === 0);
13 console.log(pares); // [2, 4]
14
15 // Encadenado: filtrar pares y multiplicar por 10
16 const resultado = numeros
17   .filter(n => n % 2 === 0)
18   .map(n => n * 10);
19 console.log(resultado); // [20, 40]
```

Donde se usa en la practica: ui.js

En `renderizarTarjetas()` se usa `datos.map()` para transformar cada objeto en HTML, y `.join("")` para unir los fragmentos en un solo string.

3: El DOM (Document Object Model)

21.1 Seleccionar elementos del HTML

Ejemplo 6 – getElementById y querySelector

```
1 // getElementById: busca por id (el mas rapido)
2 const titulo = document.getElementById('titulo-datos');
3 console.log(titulo); // <h2 id="titulo-datos">...</h2>
4 console.log(titulo.textContent); // el texto dentro del h2
5
6 // querySelector: busca con selector CSS (mas flexible)
7 const nav = document.querySelector('nav');
8 const primerEnlace = document.querySelector('nav a');
9 const btnTema = document.querySelector('#btn-tema');
10
11 // querySelectorAll: devuelve TODOS los que coinciden
12 const todosLosLinks = document.querySelectorAll('nav a');
13 console.log(todosLosLinks.length); // numero de enlaces
```

21.2 Modificar el contenido y atributos

Ejemplo 7 – Cambiar texto, HTML y atributos

```
1 const el = document.getElementById('estado-carga');
2
3 // textContent: cambia solo el texto (SEGURO contra XSS)
4 el.textContent = 'Cargando datos...';
5
6 // innerHTML: cambia el HTML interno (PELIGROSO si viene del
7 // usuario)
8 el.innerHTML = '<strong>Cargando...</strong>';
9
10 // setAttribute: cambia cualquier atributo
11 const btn = document.getElementById('btn-tema');
12 btn.setAttribute('aria-pressed', 'true');
13 btn.textContent = 'Modo claro';
14
15 // Leer un atributo
16 const valor = btn.getAttribute('aria-pressed'); // "true"
```

Donde se usa en la practica: ui.js

mostrarCarga() usa textContent para mostrar el mensaje. renderizarTarjetas() usa innerHTML pero con escapeHTML() para prevenir XSS. toggleTema() usa setAttribute para cambiar aria-pressed.

21.3 Escuchar eventos

Ejemplo 8 – addEventListener

```
1 // Escuchar clic en un boton
2 const btn = document.getElementById('btn-tema');
3 btn.addEventListener('click', () => {
4   console.log('Boton clickeado!');
5 });
6
7 // Escuchar cuando el DOM esta listo (ANTES de acceder a elementos
8   )
9 document.addEventListener('DOMContentLoaded', () => {
10   console.log('DOM listo. Ahora puedo buscar elementos.');
```

11 const titulo = document.getElementById('titulo-datos');

12 // Aqui titulo NUNCA es null

13 }

14 // Escuchar envio de formulario

15 const form = document.getElementById('formulario-pfc');

16 form.addEventListener('submit', (e) => {

17 e.preventDefault(); // Evitar que la pagina se recargue

18 console.log('Formulario enviado sin recargar!');

19 });

Por que DOMContentLoaded

Si el `<script>` esta en el `<head>`, se ejecuta ANTES de que el HTML exista. `getElementById` devolveria `null` porque el elemento aun no existe. `DOMContentLoaded` espera a que todo el HTML este parseado. Alternativa: poner el `<script>` al final del `<body>` con `type="module"` (que es `defer` automaticamente).

Donde se usa en la practica: app.js

Todo el codigo de `app.js` esta dentro de `document.addEventListener('DOMContentLoaded', async () =>{...})`. El listener del formulario usa `e.preventDefault()` para evitar la recarga.

4: Modulos ES6 (import / export)

22.1 export: exponer funciones de un modulo

Ejemplo 9 – export en un modulo

```
1 // archivo: js/modules/matematicas.js
2
3 // Export individual (named export)
4 export function sumar(a, b) {
5     return a + b;
6 }
7
8 export function restar(a, b) {
9     return a - b;
10 }
11
12 // Variable exportada
13 export const PI = 3.14159;
14
15 // Funcion privada (sin export = no se puede importar)
16 function logInterno(msg) {
17     console.log('[mate] ${msg}');
18 }
```

22.2 import: usar funciones de otro modulo

Ejemplo 10 – import en el archivo principal

```
1 // archivo: js/app.js
2
3 // Importar funciones especificas
4 import { sumar, restar, PI } from './modules/matematicas.js';
5
6 console.log(sumar(3, 5)); // 8
7 console.log(PI); // 3.14159
8
9 // NO se puede acceder a logInterno() porque no tiene export
```

Requisito en el HTML

Para que import/export funcionen, el script debe tener type="module":

```
<script type="module" src="js/app.js"></script>
```

Sin type="module" el navegador lanza: SyntaxError: Cannot use import statement outside a module.

Donde se usa en la practica: Toda la arquitectura del Paso 4

El PFC tiene 3 modulos: api.js exporta obtenerDatos() y enviarDatos(). ui.js exporta mostrarCarga(), renderizarTarjetas(), escapeHTML(), toggleTema(). app.js importa todo y orquesta.

5: Promesas y async/await

23.1 Que es una Promesa

Concepto

Una **Promesa** es un objeto que representa un valor que **todavía no existe** pero existira en el futuro (o fallara). Tiene 3 estados: **pending** (esperando), **fulfilled** (exito) y **rejected** (error). `fetch()` devuelve una Promesa.

Ejemplo 11 – Promesa basica con `.then()/catch()`

```
1 // fetch() devuelve una Promesa
2 // .then() se ejecuta cuando la Promesa se resuelve (exito)
3 // .catch() se ejecuta cuando la Promesa se rechaza (error)
4
5 fetch('https://jsonplaceholder.typicode.com/users/1')
6 .then(respuesta => respuesta.json())
7 .then(usuario => console.log(usuario.name))
8 .catch(error => console.error('Fallo:', error));
9
10 // Resultado: "Leanne Graham"
```

23.2 async/await: Promesas legibles

Ejemplo 12 – Lo mismo pero con `async/await`

```
1 // async hace que la funcion devuelva una Promesa
2 // await pausa la ejecucion hasta que la Promesa se resuelva
3 // SOLO se puede usar await dentro de una funcion async
4
5 async function cargarUsuario() {
6   const respuesta = await fetch(
7     'https://jsonplaceholder.typicode.com/users/1'
8   );
9   const usuario = await respuesta.json();
10  console.log(usuario.name); // "Leanne Graham"
11 }
12
13 cargarUsuario();
```

Por que `async/await` es mejor que `.then()`

Comparar los ejemplos 11 y 12: hacen exactamente lo mismo, pero el ejemplo 12 se lee como codigo normal de arriba a abajo. Con `.then()` encadenado, el codigo se vuelve dificil de leer cuando hay 3 o 4 operaciones asincronas seguidas (“callback hell”).

6: fetch – hablar con el servidor

24.1 GET: obtener datos

Ejemplo 13 – fetch GET con verificacion de error

```
1  async function obtenerUsuarios() {
2      const res = await fetch(
3          'https://jsonplaceholder.typicode.com/users'
4      );
5
6      // IMPORTANTE: fetch NO lanza error con 404 o 500
7      // Solo lanza error si no hay internet o el dominio no existe
8      // Por eso verificamos manualmente:
9      if (!res.ok) {
10         throw new Error('Error HTTP ${res.status}');
11     }
12
13     const datos = await res.json(); // Convertir respuesta a objeto
14     JS
15     console.log('Recibidos ${datos.length} usuarios');
16     return datos;
17 }
18
19 obtenerUsuarios();
20 // Recibidos 10 usuarios
```

Donde se usa en la practica: `api.js` — funcion `obtenerDatos()`

Este es exactamente el patron de la funcion `obtenerDatos()` del modulo `api.js` de la practica.

24.2 POST: enviar datos

Ejemplo 14 – fetch POST con JSON

```
1  async function crearUsuario(datos) {
2    const res = await fetch(
3      'https://jsonplaceholder.typicode.com/users',
4      {
5        method: 'POST', // Verbo HTTP
6        headers: { 'Content-Type': 'application/json' }, // Tipo
7        body: JSON.stringify(datos), // Objeto -> texto
8        // JSON
9      }
10   );
11
12   if (!res.ok) {
13     throw new Error('Error al crear: ${res.status}');
14   }
15
16   return res.json();
17 }
18
19 // Usar:
20 crearUsuario({ nombre: "Ana", email: "ana@uteq.edu.ec" })
   .then(resp => console.log('Creado:', resp));
```

Donde se usa en la practica: api.js — funcion enviarDatos()

Este es el patron de enviarDatos() que se usa cuando el formulario se envia.

7: try/catch/finally – manejar errores

Ejemplo 15 – Los 4 estados que siempre hay que manejar

```
1  async function cargarDatos() {
2      // ESTADO 1: Cargando
3      console.log('Cargando...');
4
5      try {
6          const res = await fetch(
7              'https://jsonplaceholder.typicode.com/users?_limit=3'
8          );
9          if (!res.ok) throw new Error('HTTP ${res.status}');
10         const datos = await res.json();
11
12         // ESTADO 2: Exito
13         if (datos.length === 0) {
14             // ESTADO 3: Vacio (exito pero sin datos)
15             console.log('No hay datos.');
```

Resultado esperado

Cargando...

Datos: ["Leanne Graham", "Ervin Howell", "Clementine Bauch"]

Fin. Ocultar indicador de carga.

Donde se usa en la practica: app.js — funcion cargarDatos()

Este patron de 4 estados es exactamente la funcion `cargarDatos()` de `app.js`. La practica usa `mostrarCarga()` en lugar de `console.log` y `renderizarTarjetas()` en lugar de `console.log` del array.

8: Manipular el DOM con datos de la API

26.1 Renderizar datos como HTML

Ejemplo 16 – Convertir un array de objetos en tarjetas HTML

```
1 // Funcion anti-XSS: OBLIGATORIA antes de insertar texto en
   innerHTML
2 function escapeHTML(texto) {
3   return String(texto)
4     .replace(/&/g, '&amp;')
5     .replace(/</g, '&lt;')
6     .replace(/>/g, '&gt;')
7     .replace(/"/g, '&quot;');
8 }
9
10 // Datos que vinieron de la API:
11 const usuarios = [
12   { id: 1, name: "Leanne Graham", email: "l@mail.com" },
13   { id: 2, name: "Ervin Howell", email: "e@mail.com" },
14 ];
15
16 // Convertir cada objeto en HTML con .map() + template literal
17 const html = usuarios.map(u => `
18   <article class="tarjeta">
19     <h3>${escapeHTML(u.name)}</h3>
20     <p>${escapeHTML(u.email)}</p>
21     <p><small>ID: ${u.id}</small></p>
22   </article>
23 `).join(''); // .join('') une los fragmentos en 1 solo string
24
25 // Insertar en el DOM
26 const contenedor = document.getElementById('contenedor-datos');
27 contenedor.innerHTML = html;
```

Por que escapeHTML es obligatorio

Si un usuario malicioso guardara como nombre `<script>alert('XSS')</script>`, sin `escapeHTML` ese script se ejecutaria en el navegador de todos los demas usuarios. Con `escapeHTML`, los `<` y `>` se convierten en `<` y `>`, y el navegador muestra el texto en lugar de ejecutarlo.

Donde se usa en la practica: ui.js — funcion `renderizarTarjetas()`

Este ejemplo es exactamente lo que hace `renderizarTarjetas()` en el Paso 4.

9: localStorage y modo oscuro

Ejemplo 17 – localStorage: guardar y leer datos persistentes

```
1 // Guardar un valor (persiste al cerrar el navegador)
2 localStorage.setItem('tema-pfc', 'oscuro');
3
4 // Leer un valor (devuelve null si no existe)
5 const tema = localStorage.getItem('tema-pfc');
6 console.log(tema); // "oscuro"
7
8 // Eliminar un valor
9 localStorage.removeItem('tema-pfc');
```

Ejemplo 18 – Toggle de modo oscuro completo

```
1 function toggleTema() {
2     const html = document.documentElement; // = <html>
3     const actual = html.getAttribute('data-tema');
4     const nuevo = actual === 'oscuro' ? 'claro' : 'oscuro';
5
6     // Cambiar el atributo data-tema en <html>
7     html.setAttribute('data-tema', nuevo);
8
9     // Guardar la preferencia
10    localStorage.setItem('tema-pfc', nuevo);
11
12    // Actualizar el texto del boton
13    const btn = document.getElementById('btn-tema');
14    btn.textContent = nuevo === 'oscuro' ? 'Modo claro' : 'Modo
15        oscuro';
16    btn.setAttribute('aria-pressed', nuevo === 'oscuro');
17 }
18
19 // Al cargar la pagina: restaurar la preferencia guardada
20 function cargarTema() {
21     const guardado = localStorage.getItem('tema-pfc');
22     if (guardado) {
23         document.documentElement.setAttribute('data-tema', guardado);
24     }
25 }
```

Donde se usa en la practica: ui.js — funciones toggleTema() y cargarTema()

Estas dos funciones son exactamente las que van en ui.js. El CSS en variables.css tiene la regla [data-tema="oscuro"] que cambia los colores automaticamente cuando el atributo cambia.

10: Formularios con JavaScript

Ejemplo 19 – Validar y enviar un formulario sin recargar

```
1 function configurarFormulario() {
2   const form = document.getElementById('formulario-pfc');
3   if (!form) return; // Proteccion si el formulario no existe
4
5   form.addEventListener('submit', async (e) => {
6     // 1. Evitar que la pagina se recargue
7     e.preventDefault();
8
9     // 2. Verificar validacion nativa del HTML
10    if (!form.checkValidity()) {
11      form.reportValidity(); // Muestra los mensajes del navegador
12      return; // No continuar si hay errores
13    }
14
15    // 3. Extraer los datos del formulario como objeto
16    const datos = Object.fromEntries(new FormData(form));
17    console.log('Datos del formulario:', datos);
18    // { nombre: "Ana", email: "ana@...", carrera: "sw", ... }
19
20    // 4. Enviar a la API
21    try {
22      const resp = await enviarDatos('users', datos);
23      console.log('Enviado:', resp);
24      form.reset(); // Limpiar el formulario
25
26      // 5. Mostrar confirmacion accesible
27      const estado = document.getElementById('estado-carga');
28      estado.textContent = 'Registro exitoso.';
29      setTimeout(() => { estado.textContent = ''; }, 4000);
30    } catch (error) {
31      console.error('Error al enviar:', error);
32    }
33  });
34 }
```

Object.fromEntries + FormData

`new FormData(form)` captura todos los campos del formulario. `Object.fromEntries()` convierte esos pares clave-valor en un objeto JavaScript plano, listo para enviar como JSON con `JSON.stringify()`.

Donde se usa en la practica: app.js — funcion configurarFormulario()

Esta funcion es exactamente la que va en `app.js`. Conecta el formulario HTML con la API sin recargar la pagina.

FINAL: Todo junto – el codigo de la practica

Verificacion final

Si entendiste los 19 ejemplos anteriores, puedes leer y escribir cada linea del Paso 4 de la practica. El codigo final es la **combinacion** de todos los conceptos anteriores:

#	Concepto	Donde se usa en la practica	Ejemplo en este documento
1	<code>const / let</code>	Todas las variables	Ejemplo 1
2	Template literals	HTML de tarjetas, mensajes de error	Ejemplo 2
3	Objetos y arrays	Datos de la API	Ejemplo 3
4	Arrow functions	Callbacks de <code>map</code> , <code>forEach</code> , <code>listeners</code>	Ejemplo 4
5	<code>.map()</code> y <code>.join()</code>	<code>renderizarTarjetas()</code>	Ejemplo 5
6	<code>getElementById</code>	Acceder a elementos del DOM	Ejemplo 6
7	<code>textContent / innerHTML</code>	<code>mostrarCarga()</code> , <code>renderizar()</code>	Ejemplo 7
8	<code>addEventListener</code>	Click del boton, submit del form	Ejemplo 8
9	<code>DOMContentLoaded</code>	Punto de entrada de <code>app.js</code>	Ejemplo 8
10	<code>export / import</code>	Arquitectura de 3 modulos	Ejemplos 9 y 10
11	<code>async / await</code>	<code>obtenerDatos()</code> , <code>cargarDatos()</code>	Ejemplo 12
12	<code>fetch()</code> GET	<code>obtenerDatos()</code>	Ejemplo 13
13	<code>fetch()</code> POST	<code>enviarDatos()</code>	Ejemplo 14
14	<code>try/catch/finally</code>	4 estados de carga	Ejemplo 15
15	<code>escapeHTML()</code>	Prevencion XSS	Ejemplo 16
16	<code>.map().join() + HTML</code>	<code>renderizarTarjetas()</code>	Ejemplo 16
17	<code>localStorage</code>	Persistir el tema oscuro	Ejemplo 17
18	Toggle de tema	<code>toggleTema()</code> / <code>cargarTema()</code>	Ejemplo 18
19	<code>FormData + preventDefault</code>	<code>configurarFormulario()</code>	Ejemplo 19

Estas listo

Si puedes explicar cada fila de esta tabla, puedes implementar el Paso 4 completo de la practica sin copiar y pegar. Abre la guia de la PE U1, ve al Paso 4, y veras que cada linea de codigo corresponde a uno de estos 19 ejemplos.

Integracion – Como trabajan juntos HTML, CSS y JS

Flujo completo: del HTML al resultado final

Concepto: separacion de responsabilidades

Las 3 tecnologias tienen roles distintos y se comunican a traves de **atributos del HTML**:

- **HTML** define la estructura y el contenido. Usa `class` para conectar con CSS y `id` para conectar con JavaScript.
- **CSS** define la apariencia. Lee las `class` del HTML y aplica estilos. Nunca modifica el contenido ni el comportamiento.
- **JavaScript** define el comportamiento. Lee `id` del HTML para encontrar elementos, modifica su `textContent`, `innerHTML` y atributos, y escucha eventos como `click` y `submit`.

Los 3 archivos estan separados: `index.html`, `css/main.css` y `js/app.js`. Se conectan con `<link>` y `<script>` en el HTML.

INT-1 – Mapa de conexiones entre los 3 archivos

Este ejemplo muestra como un mismo elemento HTML es tocado por CSS (apariencia) y por JS (comportamiento) a traves de sus atributos `class` e `id`.

```
1      <!-- index.html -->
2      <!-- class="btn-primario" -> CSS le da el estilo -->
3      <!-- id="btn-tema" -> JS le agrega el evento -->
4      <button id="btn-tema" class="btn-primario"
5      aria-pressed="false">
6      Modo oscuro
7      </button>
8
9      <!-- class="tarjeta" -> CSS le da forma y animacion -->
10     <!-- role="listitem" -> ARIA para lectores de pantalla -->
11     <!-- JS genera este HTML dinamicamente con innerHTML -->
12     <article class="tarjeta" role="listitem">
13     <h3>Leanne Graham</h3>
14     <p>leanne@mail.com</p>
15     </article>
16
17     <!-- class="campo-form" -> CSS lo organiza vertical -->
18     <!-- id="formulario-pfc" -> JS captura el submit -->
19     <form id="formulario-pfc">
20     <div class="campo-form">
21     <label for="nombre">Nombre</label>
22     <input type="text" id="nombre" class="input-field">
23     </div>
24     </form>
```

Arquitectura de archivos del PFC

INT-2 – Estructura de archivos y quien usa que

```
frontend/  
index.html      <- Estructura (Paso 2)  
css/  
variables.css   <- Variables :root (Paso 3)  
main.css        <- Estilos visuales (Paso 3)  
js/  
app.js          <- Orquestador (Paso 4)  
modules/  
api.js          <- fetch a la API (Paso 4)  
ui.js           <- Manipular el DOM (Paso 4)  
  
Conexiones:  
index.html --<link>--> css/main.css --@import--> css/variables.css  
index.html --<script>--> js/app.js --import--> js/modules/api.js  
--import--> js/modules/ui.js  
  
CSS lee del HTML:  class="tarjeta", class="btn-primario", [data-  
                    tema]  
JS lee del HTML:   id="btn-tema", id="formulario-pfc", id="  
                  contenedor-datos"  
JS modifica HTML:  textContent, innerHTML, setAttribute, classList
```

Ejemplo completo: boton de tema (HTML + CSS + JS)

INT-3a – El HTML del boton

```
1      <!-- En index.html: -->  
2      <button id="btn-tema" class="btn-primario"  
3      aria-label="Cambiar tema"  
4      aria-pressed="false">  
5      Modo oscuro  
6      </button>
```

INT-3b – El CSS del boton y del modo oscuro

```
1      /* En main.css: estilo del boton */
2      .btn-primario {
3          background: var(--color-primario);
4          color: #fff;
5          border: none;
6          border-radius: 6px;
7          cursor: pointer;
8          transition: background 150ms ease;
9      }
10     .btn-primario:hover {
11         background: var(--color-primario-hover);
12     }
13
14     /* En variables.css: modo oscuro */
15     [data-tema="oscuro"] {
16         --color-fondo: #0f172a;
17         --color-texto: #f1f5f9;
18     }
```

INT-3c – El JavaScript que conecta todo

```
1      // En ui.js:
2      export function toggleTema() {
3          const html = document.documentElement;
4          const actual = html.getAttribute('data-tema');
5          const nuevo = actual === 'oscuro' ? 'claro' : 'oscuro';
6
7          // Cambia el atributo -> CSS reacciona automaticamente
8          html.setAttribute('data-tema', nuevo);
9
10         // Guarda la preferencia para la proxima visita
11         localStorage.setItem('tema-pfc', nuevo);
12
13         // Actualiza el boton (accesibilidad)
14         const btn = document.getElementById('btn-tema');
15         btn.textContent =
16             nuevo === 'oscuro' ? 'Modo claro' : 'Modo oscuro';
17         btn.setAttribute('aria-pressed', nuevo === 'oscuro');
18     }
19
20     // En app.js:
21     import { toggleTema } from './modules/ui.js';
22     document.getElementById('btn-tema')
23     .addEventListener('click', toggleTema);
```

Flujo completo del boton de tema

1. El usuario hace clic en el boton (**HTML**).
2. JavaScript captura el evento `click` y ejecuta `toggleTema()` (**JS**).
3. `toggleTema()` cambia `data-tema="oscuro"` en el `<html>` (**JS modifica HTML**).
4. CSS detecta el selector `[data-tema="oscuro"]` y cambia las variables de color (**CSS reacciona**).
5. El body tiene `transition: background-color 300ms` y el cambio se anima suavemente (**CSS**).
6. JavaScript guarda la preferencia en `localStorage` (**JS**).
7. Al recargar, JavaScript lee `localStorage` y restaura el tema (**JS**).

Resumen: quien hace que

Tecnologia	Responsabilidad	Ejemplo en la practica
HTML	Estructura y contenido	Etiquetas semanticas, formulario, atributos ARIA, <code>class</code> e <code>id</code>
CSS	Apariencia visual	Variables, Grid, Flexbox, modo oscuro, animaciones, accesibilidad visual
JavaScript	Comportamiento	Consumo de API, renderizado dinamico, validacion, tema, eventos